Academic Audit Parsing

PDF Text Extraction for Student Course Data

Safiullah Rattar

SYSC 4907 - Carleton Smart Course Scheduler

The Problem

Input: PDF Audit from Carleton Central

- Contains complete academic history
- Audit only available as PDF download

```
Student: 101234567 Program: Software Engineering

21FA SYSC 2006 0.50 A- 21WN SYSC 2004 0.50 B+ 25WN SYSC
4907 0.50 CUR
```

Output: Structured Data

```
interface StudentAuditData {
  studentId: string
  name: string
  program: string
  completedCourses: Array<{
    courseId: string   // "SYSC2006"
    grade: string      // "A-"
  }>
  currentCourses: Array<{
    courseId: string   // "SYSC4907"
    term: string       // "25WN"
  }>
  requirements: RequirementInfo[]
}
```

Why Not OCR?

OCR (Tesseract, Vision APIs)

Mechanism: Render PDF to image, then character recognition

Problem: Probabilistic errors
- `B` (grade) vs `8` (number)
- `I` (Incomplete) vs `1` (digit)
- `0` (zero) vs `O` (letter)

Impact: A misread grade could mark a failed course as passed

PDF Text Stream Extraction

Mechanism: Extract text encoding directly from PDF binary

Advantages:
- Deterministic: exact characters
- Fast: no image processing

Tool: `pdftotext`

Challenge: PDF text streams lack structure - requires layout reconstruction

The Layout Problem

Default `pdftotext` (Column-Major)

```
Term
22FA
22FA
21SU

Course
COMP 2804
ELEC 2501
MATH 1005
```

PDF renders Column 1 first, then Column 2. “22FA” on line 2, “COMP 2804” on line 7. **No association between term and course.**

`pdftotext -layout` (Row-Major)

```
Term   Course       Credits  Grade
22FA   COMP 2804    0.50     A+
22FA   ELEC 2501    0.50     A-
21SU   MATH 1005    0.50     TR
```

The `-layout` flag uses character coordinates in the PDF to reconstruct visual positioning.

Processing Pipeline

```
// 1. Write uploaded PDF to temp file
const tmpPdfPath = path.join(os.tmpdir(), `audit-${Date.now()}.pdf`)
fs.writeFileSync(tmpPdfPath, req.file.buffer)

// 2. Extract text with layout preservation
await execAsync(`pdftotext -layout "${tmpPdfPath}" "${tmpTxtPath}"`)

// 3. Read extracted text and parse
const pdfText = fs.readFileSync(tmpTxtPath, 'utf-8')
auditData = parseAuditPDF(pdfText)

// 4. Clean up temp files
fs.unlinkSync(tmpPdfPath)
fs.unlinkSync(tmpTxtPath)
```

1. Upload

PDF file received

2. Extract

pdftotext -layout

3. Parse

Regex on plain text

Course Extraction Pattern

```
/^\s*(\d{2}(?:FA|WN|SU))\s+([A-Z]{3,4})\s+(\d{4})\s+([\d.]+)\s+([A-Z+\-]+)(?:\s+(?:RP|CUR|SAT|TR|UNS|IP|DEF))?/gm
```

Pattern	Purpose	Example
(\d{2}(?:FA|WN|SU))	Term: 2-digit year + semester	“22FA”
([A-Z]{3,4})	Department code	“SYSC”
(\d{4})	Course number	“3303”
([\d.]+)	Credits	“0.50”
([A-Z+\-]+)	Grade with modifiers	“A-”
(?:RP|CUR|SAT|...)?	Optional status flags	“CUR”, “DEF”

Processing Matches

```
const matches = Array.from(pdfText.matchAll(courseLinePattern))

for (const match of matches) {
  const term = match[1]           // "22FA"
  const courseCode = match[2]     // "SYSC"
  const courseNum = match[3]      // "3303"
  const courseId = courseCode + courseNum  // "SYSC3303" (normalized)
  const grade = match[5]          // "A-"
  const fullMatch = match[0]      // Full line for flag detection
}
```

Why `matchAll` with `/gm`?

- Global flag: finds all matches
- Multiline flag: ^ matches each line start
- Returns iterator with captured groups
- Processes entire audit in single pass

Course ID Normalization PDF has `"SYSC 3303"` (with space)

We store `"SYSC3303"` (no space)

Matches our database format for lookups.

Grade Classification

```
const isCurrent = fullMatch.includes('CUR') || grade === 'CUR'

if (isCurrent && !seenCurrent.has(courseId)) {
  currentCourses.push({ courseId, term })
  seenCurrent.add(courseId)
} else if (grade && grade !== 'CUR' && grade !== 'UNS' &&
           grade !== 'F' && grade !== 'X' && grade !== 'NR' &&
           !seenCompleted.has(courseId)) {
  completedCourses.push({ courseId, grade })
  seenCompleted.add(courseId)
}
```

Completed

A+ through D- TR (Transfer) SAT (Satisfied)

Current/Deferred (CUR, DEF)

Currently enrolled (CUR) Deferred (DEF) - grade pending Stored with term Counts as “course taken” but no grade yet

Filtered Out

F (Failed) X (Forfeited) UNS, NR

Why Separate Current vs Completed?

Both Satisfy Prerequisites

Scenario: Planning courses for next term

- Currently taking SYSC 3303 (CUR)
- Want to take SYSC 4406 next term
- SYSC 4406 requires SYSC 3303

Result: Prerequisite satisfied

Both completed and current courses count.

But Grade Requirements Differ

Some prerequisites require minimum grade:

“SYSC 3303 with C- or higher”

- **Completed:** Has grade, can verify
- **Current:** No grade yet, cannot verify

Result: Current courses satisfy “course taken” but not grade requirements.

Handling Repeated Courses

```
const seenCompleted = new Set<string>()
const seenCurrent = new Set<string>()

// Check before adding:
if (!seenCompleted.has(courseId)) {
  completedCourses.push({ courseId, grade })
  seenCompleted.add(courseId)
}
```

Two Sets: same course can appear as completed (old attempt) and current (retaking).

Example: Retaken Course

PDF contains:

```
21FA  SYSC 2004  0.50  F
22FA  SYSC 2004  0.50  B+
```

Processing:

1. F grade → filtered out
2. B+ grade → added

Result:

```
completedCourses: [
  { courseId: "SYSC2004", grade: "B+" }
]
```

Student Info Extraction

```
// Student ID: first 9-digit number in document
const studentIdMatch = pdfText.match(/(\d{9})/)
const studentId = studentIdMatch ? studentIdMatch[1] : ''

// Name: "LastName, FirstName" -> "FirstName LastName"
const nameMatch = pdfText.match(/([A-Z][a-z]+),\s+([A-Z][a-z]+)/)
const name = nameMatch ? `${nameMatch[2]} ${nameMatch[1]}` : ''

// Program: text after "Bachelor of Engineering -"
const programMatch = pdfText.match(/Bachelor of Engineering\s*-\s*([A-Za-z\s]+)/i)
const program = programMatch ? programMatch[1].trim() : ''
```

Field	Pattern	Example
Student ID	/(\d{9})/	“101234567”
Name	/([A-Z][a-z]+),\s+([A-Z][a-z]+)/	“Smith, John” → “John Smith”
Program	/Bachelor of Engineering\s*-\s*(.+)/i	“Software Engineering”

How Parsed Data Is Used

Prerequisite Validation

User selects a course to add to schedule.

System checks:
- Is required course in `completedCourses`?
- Or in `currentCourses`?
- Does grade meet minimum requirement?

Returns: eligible or missing prerequisites

Schedule Generation

User uploads audit, selects courses.

System uses parsed data to:
- Filter courses already completed
- Validate prerequisites for selections
- Generate conflict-free schedules

Audit Parsing Summary

Technical Approach

1. **PDF text extraction** via `pdftotext -layout`
   - Preserves column alignment
   - Deterministic (not OCR)
2. **Regex pattern matching**
   - Course lines: term, code, credits, grade
   - Student info: ID, name, program
3. **Grade classification**
   - Filters failures (F, X, UNS, NR)
   - Separates current from completed
   - Deduplicates retaken courses

Key Design Decisions

Choice	Rationale
pdftotext	Deterministic vs OCR
-layout flag	Preserves row structure
Regex	Fast, no dependencies
CUR separate	Grade req validation
Set dedup	Handles retakes
Filter F/X	Only passing counts

Course Search: The Problem

Naive Implementation

User types “SYSC4001” in search box

Without optimization:

- S → API call → DB query
- SY → API call → DB query
- SYS → API call → DB query
- SYSC → API call → DB query
- … (8 keystrokes = 8 queries)

Why This Matters

Database load:

- Each query scans courses table
- Joins with sections table
- Multiplied by concurrent users

User experience:

- Results flicker with each response
- Wasted bandwidth

Three Optimization Strategies

1. Minimum Length

Don’t search for queries < 2 characters.

“S” matches too many courses.

```
if (query.length < 2)
  return []
```

2. Debouncing

Wait 300ms after typing stops before API call.

```
debounceRef.current =
  setTimeout(() => {
    performSearch(query)
  }, 300)
```

3. Caching

Store results in memory for 5 minutes.

```
cache.set(query, {
  results,
  timestamp
})
```

Debouncing Implementation

```
const DEBOUNCE_MS = 300
const debounceRef = useRef<ReturnType<typeof setTimeout> | null>(null)

const handleSearch = (query: string) => {
  setSearchQuery(query)

  // Clear previous timer (user still typing)
  if (debounceRef.current) {
    clearTimeout(debounceRef.current)
  }

  if (query.trim().length < 2) {
    setSearchResults([])
    return
  }

  // Only fires if user stops typing for 300ms
  debounceRef.current = setTimeout(() => {
    performSearch(query)
  }, DEBOUNCE_MS)
}
```

Each keystroke clears the previous timer and starts a new one. Search only executes after 300ms pause. “SYSC4001” typed quickly: 8 keystrokes → 1 API call.

Caching Implementation

```
// In-memory cache (lives in browser tab's RAM)
const searchCache = new Map<string, { results: Course[]; timestamp: number }>()
const CACHE_TTL_MS = 5 * 60 * 1000  // 5 minutes

export async function searchCourses(query: string): Promise<Course[]> {
  const normalizedQuery = query.trim().toLowerCase()

  // Check cache first
  const cached = searchCache.get(normalizedQuery)
  if (cached && Date.now() - cached.timestamp < CACHE_TTL_MS) {
    return cached.results  // Cache hit - skip API call
  }

  const results = await fetch(...)  // Cache miss - make API call

  searchCache.set(normalizedQuery, { results, timestamp: Date.now() })
  return results
}
```

Where is it stored? In-memory `Map` in the browser tab. Cleared on page refresh or tab close. Not localStorage.

Why 5 minutes TTL? Course data doesn't change during a session. 5 min covers typical usage without stale data risk.

Backend Search Endpoint

```
router.get('/search', async (req: Request, res: Response) => {
  const { q, limit = '20' } = req.query
  const limitNum = Math.min(parseInt(limit as string) || 20, 100)

  const courses = await prisma.course.findMany({
    where: {
      OR: [
        { id: { contains: q.toUpperCase(), mode: 'insensitive' } },
        { title: { contains: q, mode: 'insensitive' } }
      ],
      sections: { some: {} }  // Only courses with available sections
    },
    take: limitNum,
    orderBy: { id: 'asc' }
  })

  res.json({ results: courses, count: courses.length })
})
```

OR clause: matches course code or title. `sections: { some: {} }` filters out courses with no sections (can't schedule them). Default limit 20, max 100.

Optimization Results

Scenario	Before	After
User types “SYSC4001” (8 chars)	8 requests	1 request
User searches “SYSC” twice	2 requests	1 request
User types “S” then deletes	1 request	0 requests
User types fast, pauses, continues	N requests	2 requests

Combined Effect Typical session: 50+ keystrokes, 5-10 actual searches needed.

Significant reduction in API calls and server load.

Summary

- Min length: eliminates useless queries
- Debounce: waits for typing to stop
- Cache: avoids repeated queries